Binary Search

```
int binSearch(int[] a, int val) {
    int l = -1, h = a.length;
    while (l + 1 != h) {
        int m = (l + h) / 2;
        if (a[m] < val)
            l = m;
        else
            h = m;
    }
    return h;
}
```

- **Halboffene Intervalle** $[left, right)$:
- Start: `left = 0`, `right = list.size()` (ohne `-1`).
- Schleife: `while (left < right)` (nicht `<=`!).
- Mitte: `m = left + (right - left) / 2`.
- **Geschlossene Intervalle** $[left, right]$:
- Start: `left = 0`, `right = list.size() - 1`.
- Schleife: `while (left <= right)`.
- **Nützlicher Merksatz für Grenzwert-Fehler**: Wenn `right = list.size()` gewählt ist, führt `while (left <= right)` bei Element-Suchläufen außerhalb zu einem `IndexOutOfBoundsException` für `m = list.size()`.

Insertion-Sort

```
void insertionSort(int a[]) {
    int n = a.length;
    for (int i = 1; i < n; ++i) {
        int k = a[i];
        int j = i - 1;

        // Move elements of arr[0..i-1], that are
        // greater than key, to one position ahead
        // of their current position
        while (j >= 0 && a[j] > k) {
            a[j + 1] = a[j];
            j = j - 1;
        }
```

```
            a[j + 1] = k;
        }
    }
```

Selection-Sort

```
void selectionSort(int a[]) {
    int n = a.length;

    // Move boundary of unsorted subarray
    for (int i = 0; i < n - 1; i++) {
        int min_idx = i;

        // Find the minimum element in unsorted array
        for (int j = i + 1; j < n; j++) {
            if (a[j] < a[min_idx])
                min_idx = j;
        }

        // Swap the found minimum element with the first element
        int temp = a[min_idx];
        a[min_idx] = a[i];
        a[i] = temp;
    }
}
```

Beispiel einer Klasse mit Konstruktor, einem Attribut und einer Methode

```
class MyClass {
    int x;

    // Constructor
    MyClass(int value) {
        x = value;
    }

    public int getX() {
        return x;
    }
}
```

Klasse mit overloaded Methoden

```
class Calculator {

    int add(int a, int b) {
        return a + b;
    }

    int add(int a, int b, int c) {
        return a + b + c;
    }

    double add(double a, double b) {
        return a + b;
    }
}
```

Aliasing

Objekte werden in Java nicht kopiert, wenn sie einer Variable zugewiesen oder als Argument einer Methode oder einem Konstruktor übergeben werden – nur die *Referenz* wird kopiert.

Je nach dem kann dies gewollt sein, es kann aber auch zu ungewollten Effekten führen.

Dem entgegenwirken kann man entweder mit **Defensive Copying** oder **Immutability**.

Immutability

- **Unveränderliche Objekte (Immutable Classes)**:
- Deklariere Attribute als `private final`.
- **Keine Mutatoren/Setter** anbieten!
- **Verändernde Methoden (wie `plus`, `add`)**: Verändern **nie** `this`, sondern geben immer ein **neues Objekt** zurück:
  `return new Amount(this.galleons + other.galleons, ...);`

Preconditions & Exceptions

- Vorbedingungen prüfen und bei Verletzung explizit werfen:
  `if (bedinguingVerletzt) throw new IllegalArgumentException("Text");`

Klasse	Bezeichnung	Beispiel	Zuwachs $\frac{f(2n)}{f(n)}$	Wachstum
$O(1)$	Konstant	Array Access	1	sublinear
$O(\log n)$	Logarithmisch	Binäre Suche	1 + $\frac{\log 2}{\log n}$	sublinear
$O(n)$	Linear	Sequenzielle Suche	2	polynomial
$O(n \log n)$	Log-Linear	Mergesort	2 + $\frac{2 \times \log 2}{\log n}$	polynomial
$O(n^2)$	Quadratisch	Selectionsort	4	polynomial
$O(n^3)$	Kubisch	naive Matrix Multiplikation	8	polynomial
$O(2^n)$	Exponentiell	Brute Force Knapsack	2^n	exponentiell
$O(3^n)$	Exponentiell	Brute Force Coloring Problem (3 Colors)	3^n	exponentiell
$O(n!)$	Fakultät	Naive Traveling Salesman	$\prod$	exponentiell

Rechenregeln

1. **Konstante Faktoren darf man streichen.**
   es gilt zb. $k * f \in O(f)$.
   Der konstante Faktor ändert nichts an der Komplexitätsklasse. Beispielsweise gilt sowohl. $\frac{n^3}{1000} \in O(n^3)$ als auch $15 * n^3 \in O(n^3)$
2. **Bei Summen kommt es nur auf den am schnellsten wachsenden Term an.**
   Falls $f \in O(g)$ folg, dass auch $f + g \in O(g)$
   Beispielsweise $n^2 + 50n + 1'000'000 \in O(n^2)$ und $n + \log n \in O(n)$
3. **Von Polynomen ist nur der (grösste) Grad Ausschlaggebend.**
   Beispiel: $15n^5 + 20n^4 + 30n^3 + 50n^2 + 50n + 60 \in O(n^5)$
   Diese oft Nutzbare Regel ergibt sich aus der Kombination der beiden vorangegangenen Regeln.
4. **Die Basis von Logarithmen ist bei der O-Notation egal und wird weggelassen.**
   Beispiele $\log_2 n$, $\log_{10} n$ oder $\log_{42} n$ sind alle in der Klasse $O(\log n)$
5. **Exponentialfunktionen mit grösserer Basis wachsen schneller.**
   Das bedeutet beispielweise $3^n \in O(2^n)$ aber $2^n \in O(3^n)$
   Exponentialfunktionen sind die Umkehrfunktionen von Logarithmen. Im Gegensatz zu diesen sind bei Exponentialfunktionen die Basen für die asymptotische Komplexitätsklasse relevant.
6. **Exponentialfunktionen wachsen schneller als jedes Polynom.**
   $b^n \in O(n^k)$ für beliebige Wert von $b > 0$ und $k > 0$.
   Merke: Wenn die Problemgrösse n in einem Exponenten sitzt, dann handelt es sich um eine Exponentialfunktion.

7. **Produkte von Funktionen bilden jeweils eigene Klassen.**
   $n * \log(n) \in O(n * \log(n))$
   $2^n * n^2 \in O(2^n * n^2)$
   Produkte von Funktionen sind in der Komplexitätsklasse der Produkts. Das heisst, Du kannst daran nichts vereinfachen, es wird aber auch nicht komplexer: $f * g \, in O(f * g)$

- `for (int i = 0; i * i < n; ++i)` $\rightarrow \mathcal{O}(\sqrt{n})$
- `for (int j = n; j > 0; j /= 2)` $\rightarrow \mathcal{O}(\log n)$
- Schleife mit konstanten Grenzen (z. B. `i < 1000` oder `j <= 42`) $\rightarrow \mathcal{O}(1)$ (beeinflusst die Gesamtlaufzeit nicht).

ArrayList

```
import java.util.ArrayList;
```

Die folgende Tabelle zeigt, die wesentlichen Entsprechungen zwischen einem Array `arr` und einer `ArrayList` `lst` .

	Array	**ArrayList**
Typ	`Lamp[]`	`ArrayList<Lamp>`
erzeugen	`new Lamp[n]`	`new ArrayList<Lamp>(n)`
lesen	`l = arr[i]`	`l = lst.get(i)`
schreiben	`arr[i] = l`	`lst.set(i, l)`
Grösse	`arr.length`	`lst.size()`

- Erlaubt Duplikate, behält Reihenfolge bei.
- `remove("element")` entfernt das *erste* Vorkommen und gibt `boolean` zurück.
- System.out.println zeigt: `[one, two]` .

HashSet

```
private final HashSet<String> validTickets = new HashSet<>();
```

- Keine Duplikate! `add()` auf existierendes Element wird ignoriert.
- Keine garantierte Reihenfolge.
- `.remove("nicht vorhanden")` gibt `false` zurück.

HashMap

```
HashMap<String, Lamp> lampDirectory = new HashMap<>();
```

```
// fill the map with many method calls like this:
lampDirectory.put("Hallway/Ceiling-Lamp", hallway.lamp[0]);

// to find a certain `Lamp` by name use
Lamp l = lampDirectory.get("Hallway/Ceiling-Lamp");
```

- Schlüssel-Wert-Paare (Keys sind eindeutig).
- `map.put(key, val)`: Bei doppeltem Key wird der alte Wert **überschrieben**.
- `.keySet()` = Set aller Keys, `.values()` = Collection aller Werte.
- **Achtung `Map.of(...)`**: Erzeugt eine **unveränderliche (immutable)** Map.
- `map.get(key)` liefert `null`, wenn der Key nicht existiert (wirft **keine** Exception!).

Sonstiges

Wenn `equals()` in einer Klasse **nicht** überschrieben ist, verhält es sich genau wie `==`.

- **Referenzvergleich (`==`) vs. Inhaltsvergleich (`.equals()`)**:
- `==` vergleicht Speicheradressen. `null == null` ergibt `true`!
- `String`-Vergleich mit `==`: vergleicht Referenzen, nicht den Textinhalt!
- Sobald ein Ausdruck auf `null` zugreift (z.B. `null.getTitle()`), bricht das Programm mit einer Exception/Fehler ab.